problem. Construct the solution to the given problem.

This outline of recursive problem solving technique is shown below.

solver (input)

if input is small enough

construct solution

else

find sub_problems of reduced input

solutions to sub_problems = solver for each sub_problem

construct solution to the problem from

solutions to the sub_problems

Whenever we solve a problem using recursion, we have to ensure these two cases: In the recursion step, the size of the input to the recursive call is strictly smaller than the size of the given input, and there is at least one base case.

8.4.3 Recursion — Examples

Example 8.11. The recursive algorithm for length of a sequence can be written as

length (s)

-- inputs : s

-- outputs : length of s

if s has one customer -- base case

1

else

1 + length(tail(s)) -- recursion step

Example 8.12. Design a recursive algorithm to compute a^n. We constructed an iterative algorithm to compute a^n in Example 8.5. a^n can be defined recursively as

$$\mathbf{a^n} = \begin{cases} 1 & \text{if } n = 0 \\ a \times a^{n-1} & \text{otherwise} \end{cases}$$

The recursive definition can be expressed as a recursive solver for computing power(a, n).

power (a, n)

-- inputs: n is an integer , n ≥ 0

-- outputs : a^n

if n = 0 -- base case

1

else --recursion step

a × power (a, n-1)

The recursive process with solvers for calculating power(2, 5) is shown in Figure 8.8.

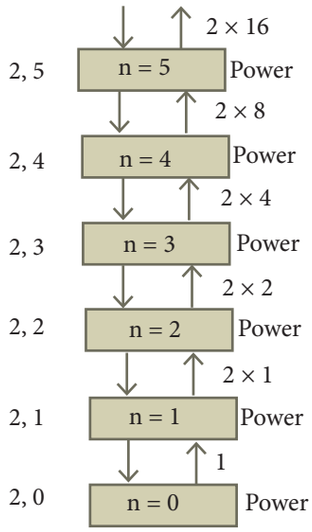


Figure 8.8: Recursive process with solvers for calculating power(2, 5)

The recursive process resulting from power(2, 5) is shown in Figure 8.9.

power (2,5)
= 2 × power (2,4)
= 2 × 2 × power(2,3)
= 2 × 2 × 2 × power(2, 2)
= 2 × 2 × 2 × 2 × power (2,1)
= 2 × 2 × 2 × 2 × 2 × power (2,0)
= 2 × 2 × 2 × 2 × 2 × 1
= 2 × 2 × 2 × 2 × 2
= 2 × 2 × 2 × 4
= 2 × 2 × 8
= 2 × 16
= 32

Figure 8.9: Recursive process for power(2, 5)

Chapter 8 Page 102-114.indd 110 3/24/2020 9:15:11 AM